
SECTION -1 PYTHON PROGRAMMING LAB

1.0	Introduction	5
1.1	Objectives	5
1.2	Setting Up a Python Development Environment	6
1.3	Instructions To Install Python	7
1.3.1	Instructions To Install Pycharm	11
1.3.2	Steps To Install The Pycharm Ide	11
1.3.3	How To Use Pycharm: First Python Program	13
1.4	List of Lab Assignments (Session Wise)	33
1.5	References/Further Readings	40

1.0 INTRODUCTION

Welcome to Section1: Python Programming Lab of the course MCSL-083 Programming and AI Lab. This section is developed to provide you hands-on experience including setting up the local environment for Python programming and fundamental concepts of the Python programming language. This section aims to provide you with practical exposure to programming using Python. Python is a high-level language and supports multiple programming paradigms, including procedural, object-oriented, and functional programming. It is a widely adopted programming language in modern software development in various domains such as data science, artificial intelligence and machine learning, robotics, automation, web development, cyber security and embedded systems etc.

Python is also known for its simple syntax, readability, and vast collection of libraries, making it a better choice for both beginners and professionals. Through this lab section, you will learn how to set up an environment for Python programming and write the Python code along with applying your problem-solving skills and logical thinking required for real-world programming. Basically, this section of MCSL-083 is based on MCS-081 course and whatever concepts and skills you will be learning in this course (MCS-081) you need to practice the lab exercises provided towards the end of this section.

1.1 OBJECTIVES

By the end of this lab section, after you go through and do all the practical exercises, you should be able to:

- Setup environment for the Python Programming language,
- Do installation of IDE (Integrated Development Environment),
- Apply the **fundamentals of Python syntax and semantics**,

- Write **Python programs** using variables, loops, conditionals, and functions,
- Implement basics of object-oriented programming in Python,
- Use data structures like lists, dictionaries, tuples, and sets,
- Handle exceptions and perform file operations in Python, and
- Use Python libraries such as Matplotlib, NumPy, Pandas.

1.2 SETTING UP A PYTHON DEVELOPMENT ENVIRONMENT

To set up a Python development environment locally, you have to follow the steps given below:

- Download Python from its official website and ensure you select the correct Python version compatible with your PC/Laptop.
- Install Python.
- Download VS Code or PyCharm IDE from its official website (you may explore and download any other IDE also as per your preference).
- Based on your choice, install the IDE (Integrated Development Environment) on your PC/Laptop.
- Write and run your first Python program: `print ("Hello, Python!")`

Visit <https://www.python.org/> and navigate to downloads

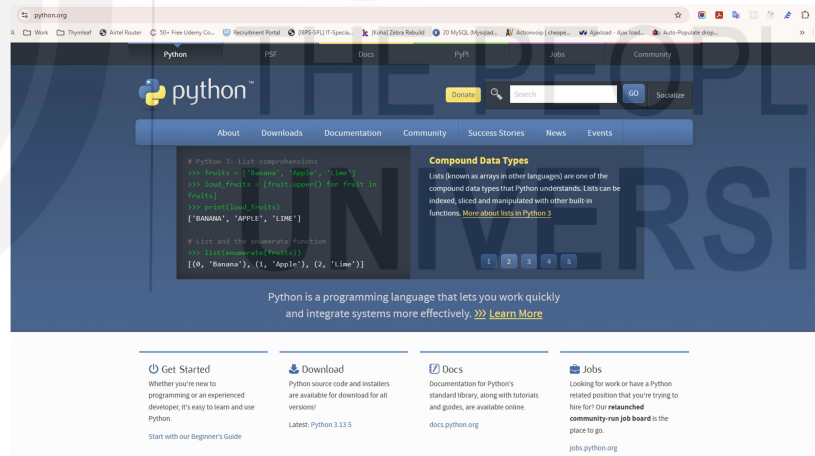


Figure 1: Official Website for Python Software download

Based on your operating system (Windows, macOS, or Linux), select the Python setup accordingly. In this lab manual for this section, we are downloading Python 3.13.5 for Windows.

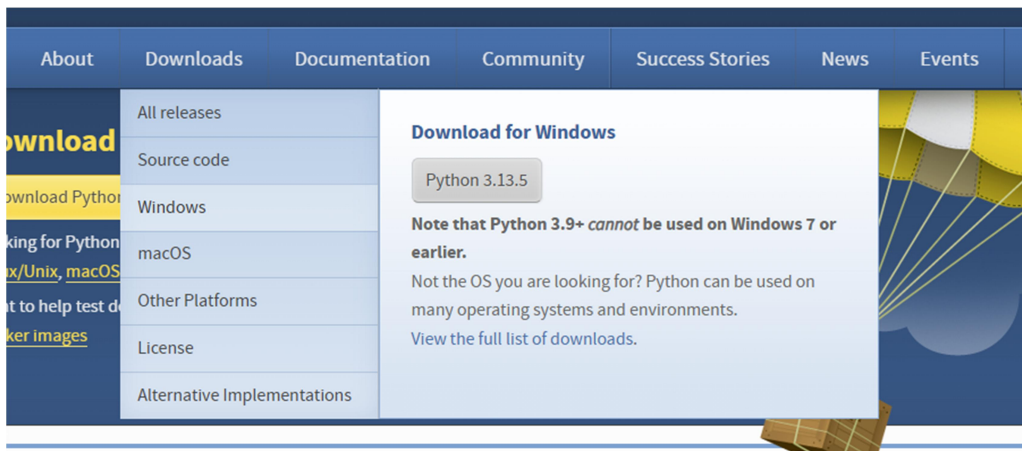


Figure 2: Various download options for Python Software

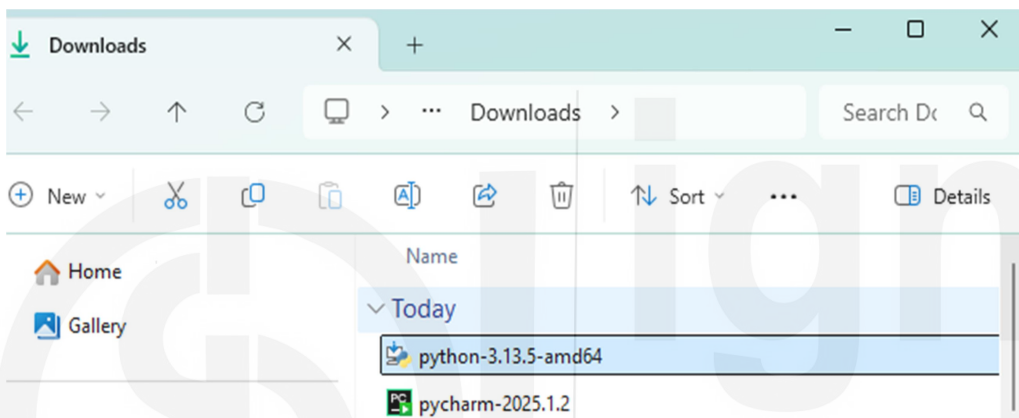


Figure 3: Setup of Python Software after downloading

1.3 INSTRUCTIONS TO INSTALL PYTHON

Once downloading is completed successfully, install the software by double-clicking on setup file. First screen of setup will appear as follows:

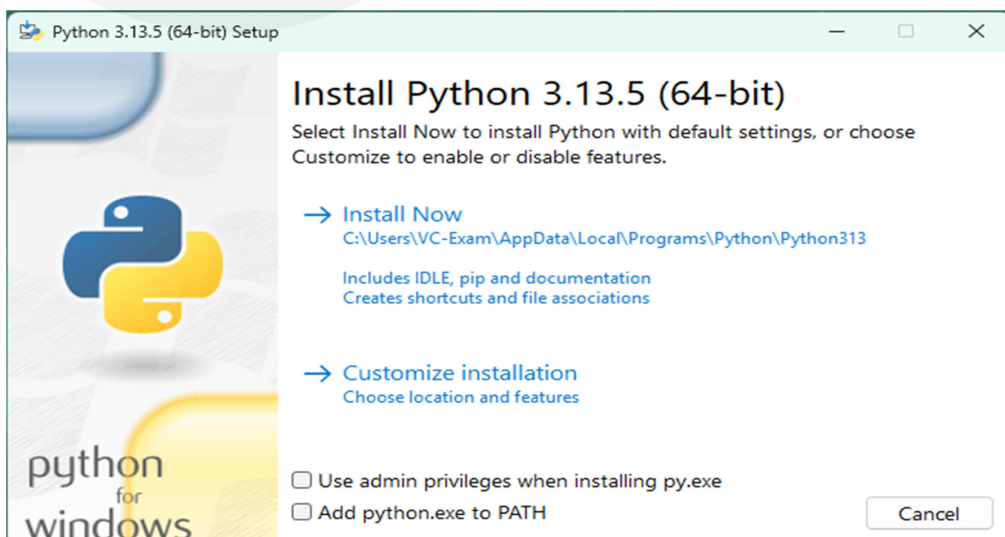


Figure 4: First Python Installation Window

Check both checkboxes before clicking to install

- ***use admin privileges when installing py.exe:*** This allows setup with admin user privileges of the operating system.
- ***Add python.exe to PATH:*** This helps to add/set the Python class path to PATH under the system variable environment.

If you want to go with the default setting, just click on **Install Now** or want a custom setting which can choose **Customize Installation**.

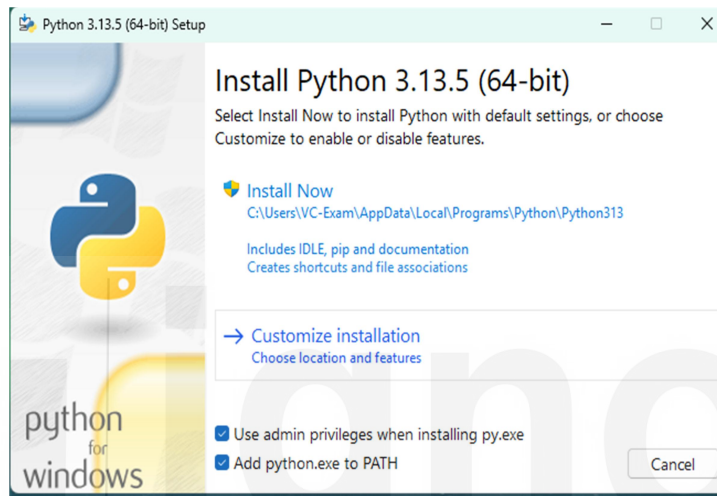


Figure 5: Setup Windows with the selection of the checkbox

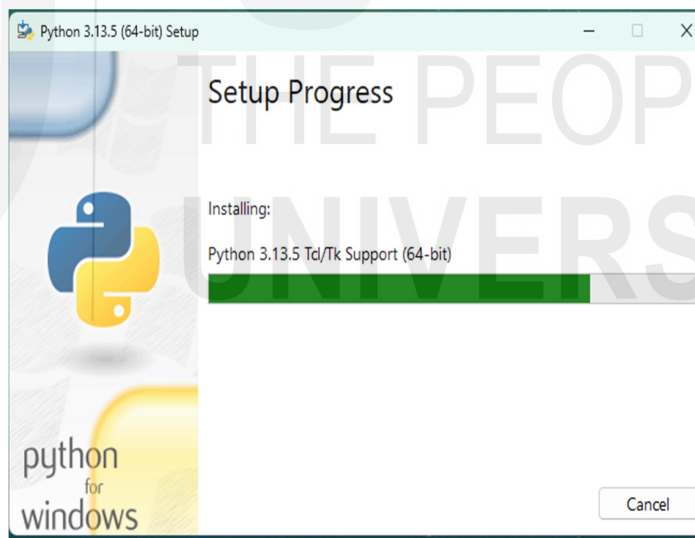


Figure 6: Python Installing in Progress

Wait till the setup is completed. After completing the following screen will appear.

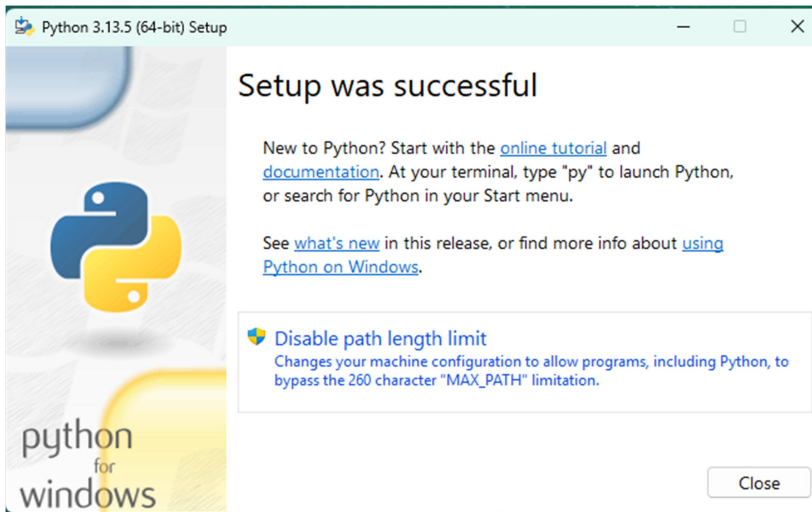


Figure 7: Installation Completion Window

Click on the button “*Disable path length limit*” to bypass the system’s default **MAX_PATH** limitation; note this is optional.

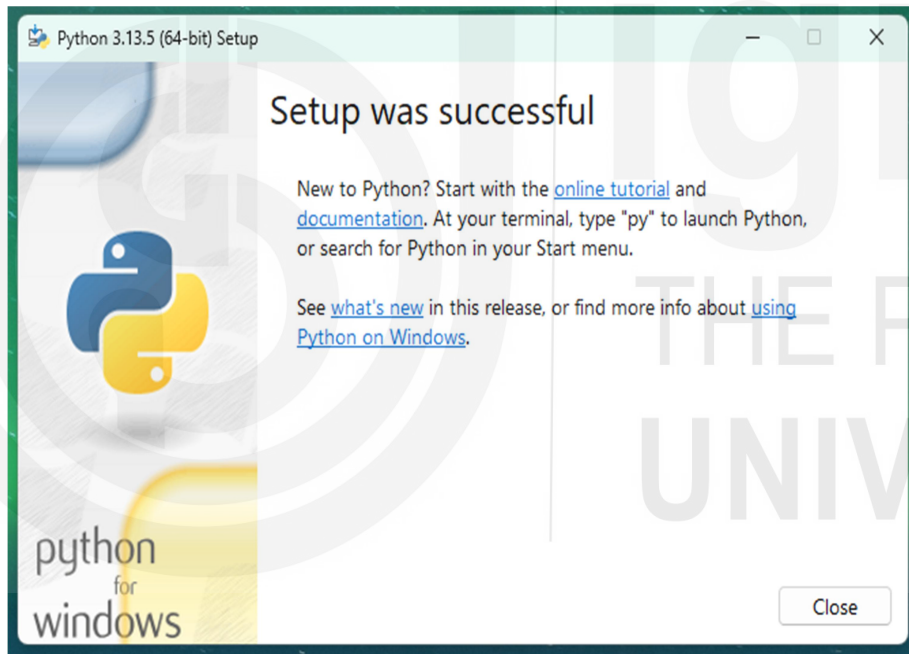


Figure 8: Final installation completion window

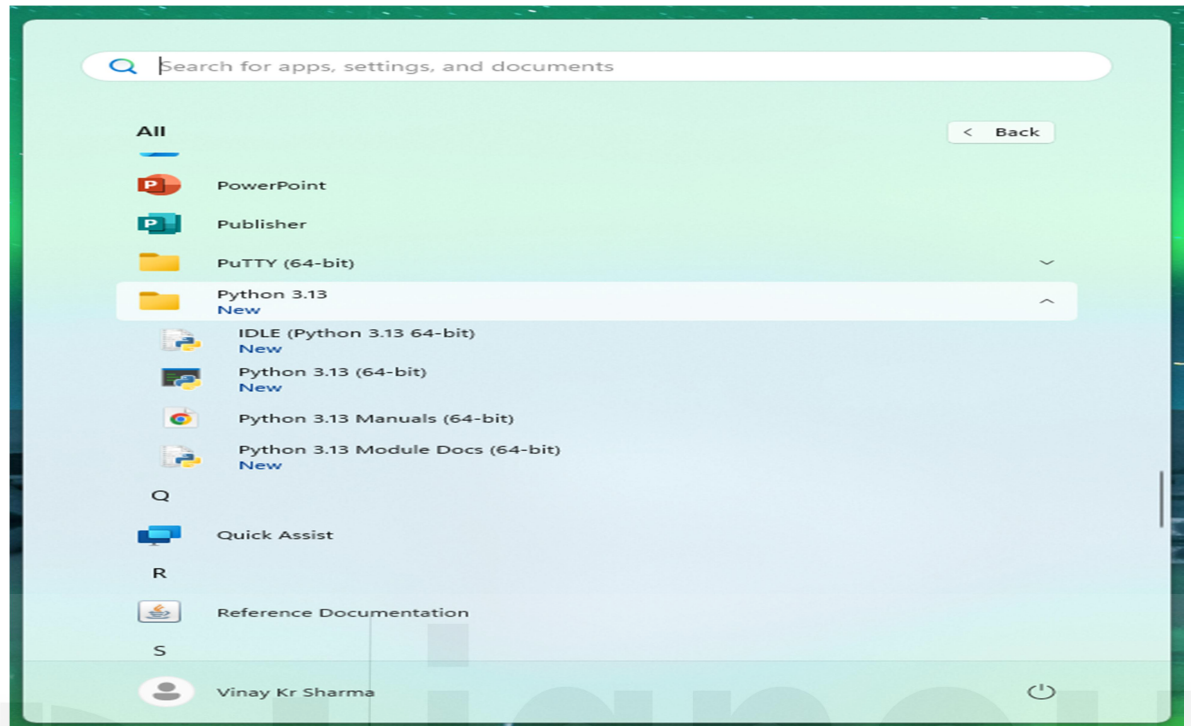


Figure 9: After installation of Python

Note, you can now see in figure 9, Python is seen in **All Programs** in the **Start menu**. Now you are ready to write your first Python code using Python's default IDLE or Shell command prompt.

Open IDLE using following steps in Windows Operating System:

Start → All Programs → Python (Version) → IDLE (Python Version)

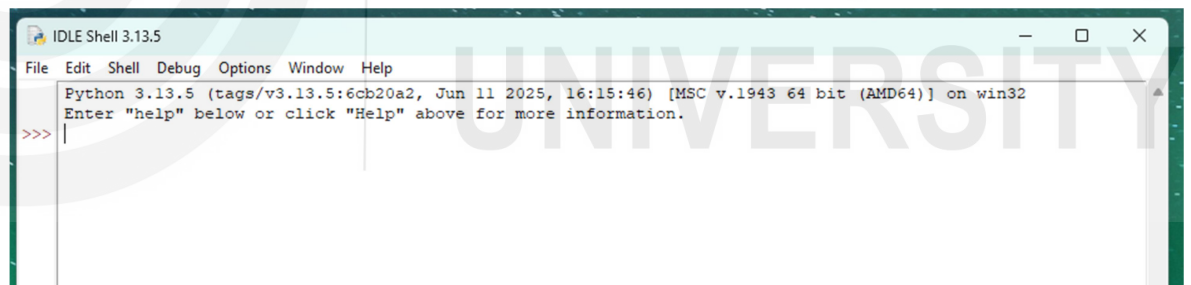


Figure 10: First look of the Python IDLE Shell Window

Write a first Hello World Program using Python in IDLE

```
print("Hello World")
```

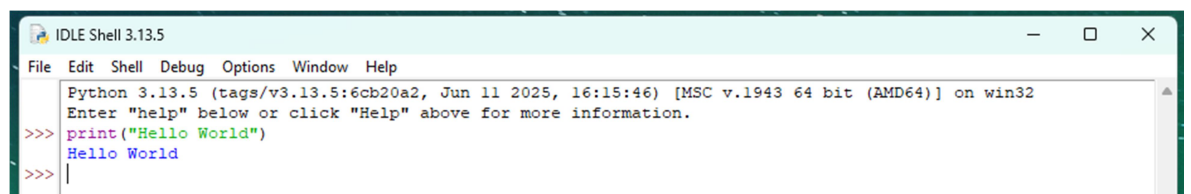
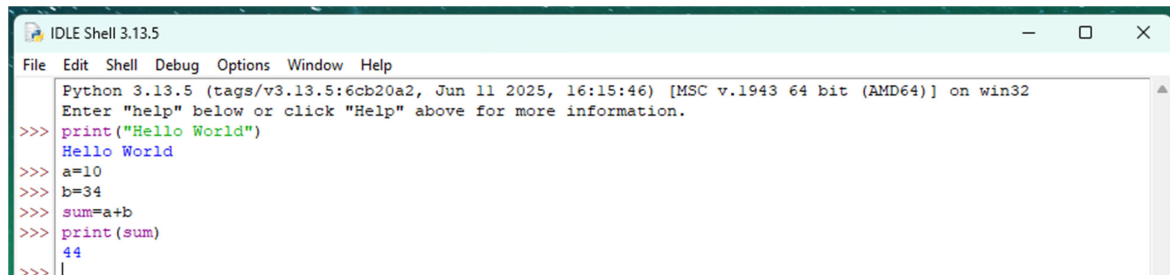


Figure 11: The output of the first program in Python

Now let us find the sum of two numbers by writing the following code:

```
a=10 ↵  
b=34 ↵  
sum=a+b ↵  
print(sum) ↵
```



```
IDLE Shell 3.13.5  
File Edit Shell Debug Options Window Help  
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>> print("Hello World")  
Hello World  
>>> a=10  
>>> b=34  
>>> sum=a+b  
>>> print(sum)  
44  
>>> |
```

Figure 12: The output of the sum of two numbers

1.3.1 Instructions to install PyCharm

PyCharm is a dedicated Python Integrated Development Environment (IDE) powered by JetBrains(<https://www.jetbrains.com/pycharm/>). PyCharm is among the first choice of top developers worldwide. It provides almost everything needed for Python application development, including out of the box to code faster and in marter environment. Whether you are an experienced developer or just a beginner of the Python programming, the JetBrains IDE for Python helps you in writing programs and developing applications. JetBrains IDE supports Python, databases, Jupyter, Git, PyTorch, TensorFlow, Hugging Face, Django, Flask, FastAPI, and many more. Out of the Box, with the seamless integration of its context-aware AI Assistant, helps you to get your work started quickly and make progress more efficiently. PyCharm equips you with all the requirements for all sorts of projects, from web development and data pipelines to ML model prototyping and data analysis. So PyCharm may be one of the obvious choices for Python IDE.

To install the JetBrains IDE for Python, you have to download it from its official website(<https://www.jetbrains.com/pycharm/>.) There are two types of editions, one community edition and another professional. You can also avail the professional edition using a student ID, following all the formalities, but for now community edition is recommended to you for learning and practicing.

1.3.2 Steps to install the PyCharm IDE

1. Download PyCharm from its official website <https://www.jetbrains.com/pycharm/>
2. Download as compatible with your operating system. Here we are using the Windows OS

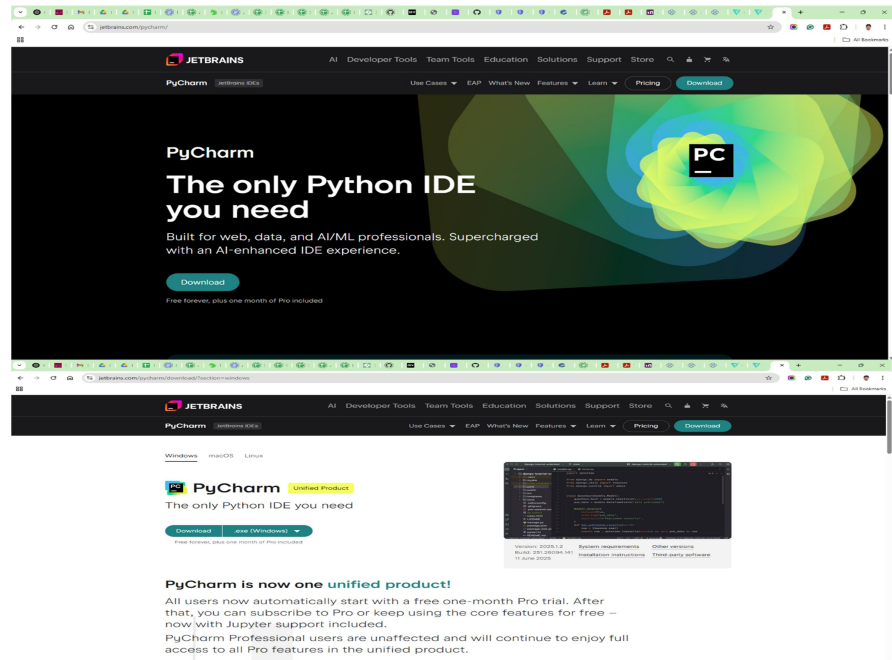


Figure 13: Official webpage of PyCharm and Download

3. Verify that the download is completed successfully, and then double-click the setup file to initiate the installation.

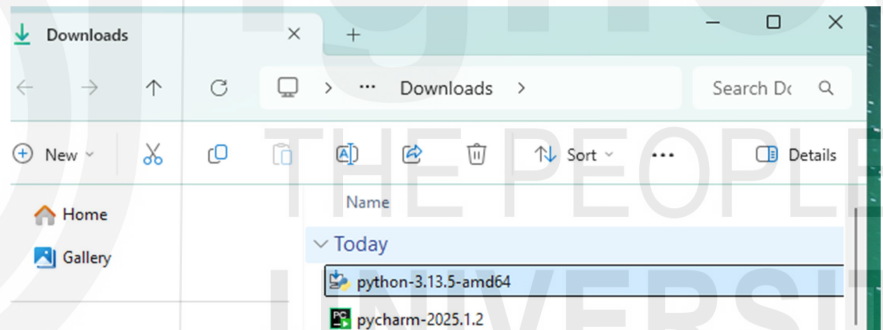
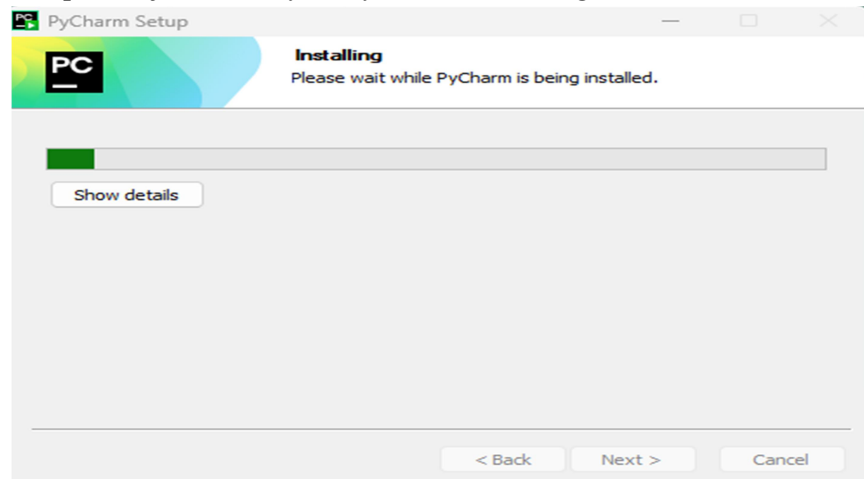


Figure 14: Setup file downloaded in the Download folder on the PC

4. Install with the default configuration and wait till it completes. Once completed, just reboot your system before using it.



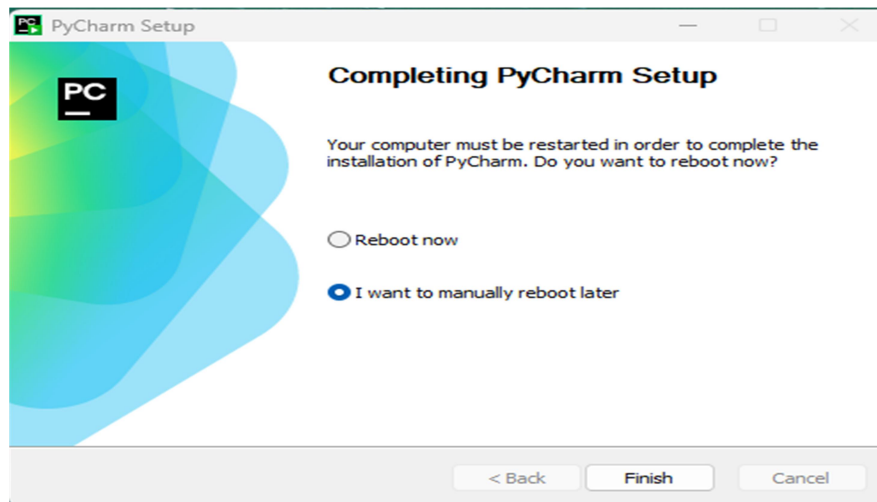


Figure 15: Installation Process of PyCharm and Completion

5. Now you are ready to use PyCharm IDE for Python programming. Open the application using the steps given in following section.

1.3.3 How to use PyCharm: First Python Program

Let us start using the PyCharm IDE for writing Python code.

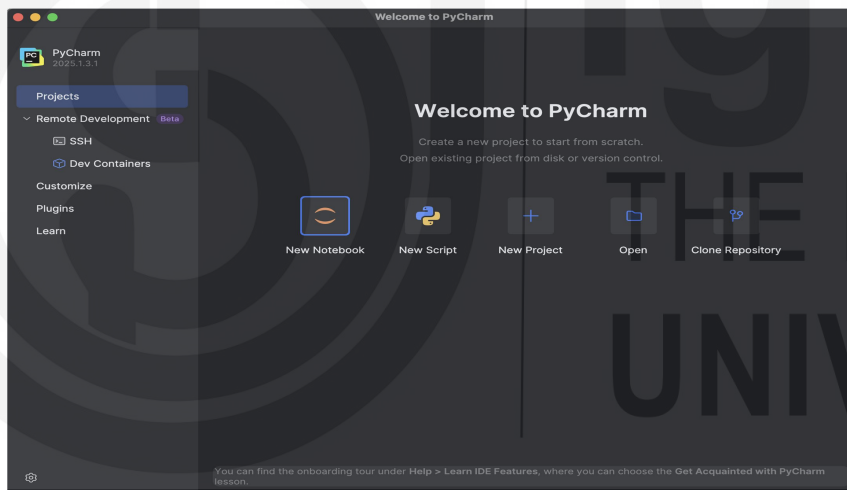


Figure 16: PyCharm Welcome Window

Click on the New Script to run your first programme and test the environment.
if you want to use Jupyter Notebook, click on New Notebook.

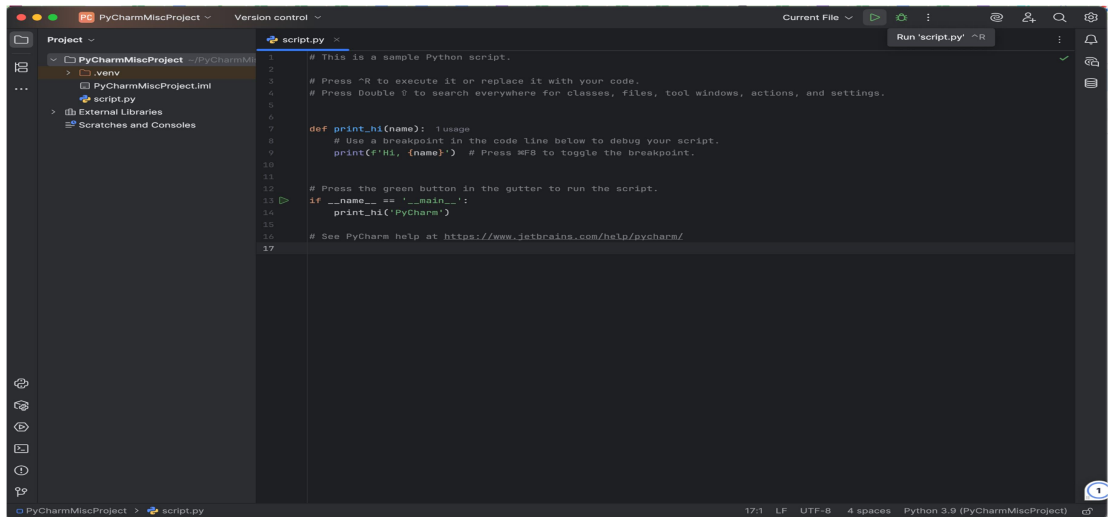


Figure 17: First look of PyCharm interface

Now click on the icon ► to run the script/project/file and the output will show in the output window bottom of the IDE as shown in the figure 18, below.

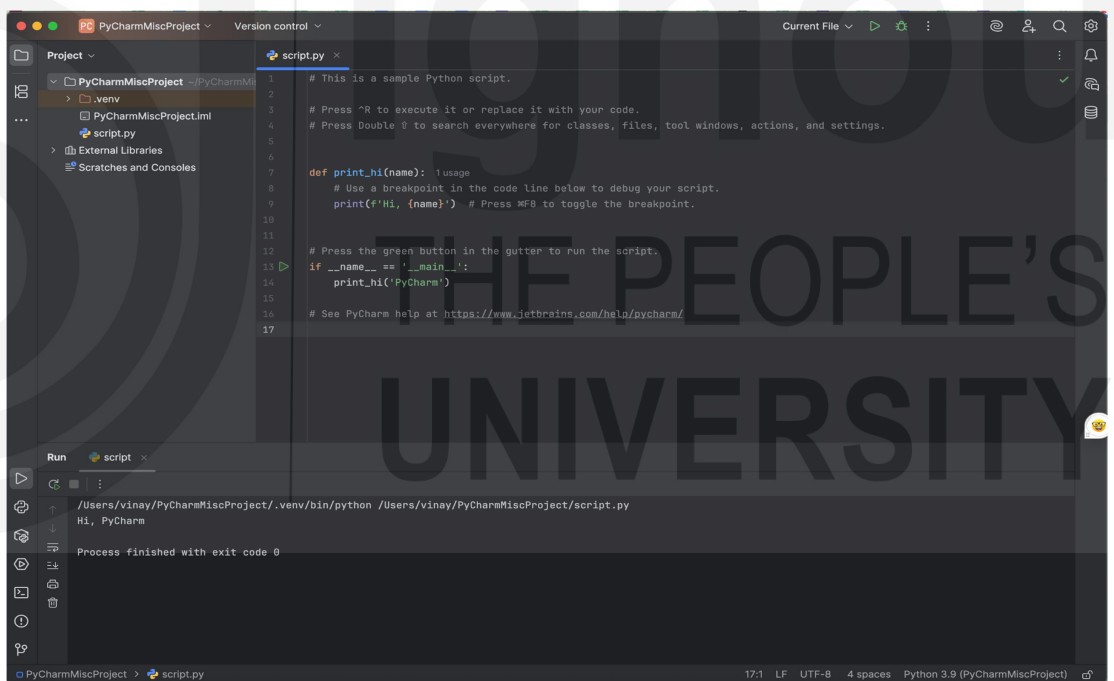


Figure 18: Python Script/Project Windows with output.

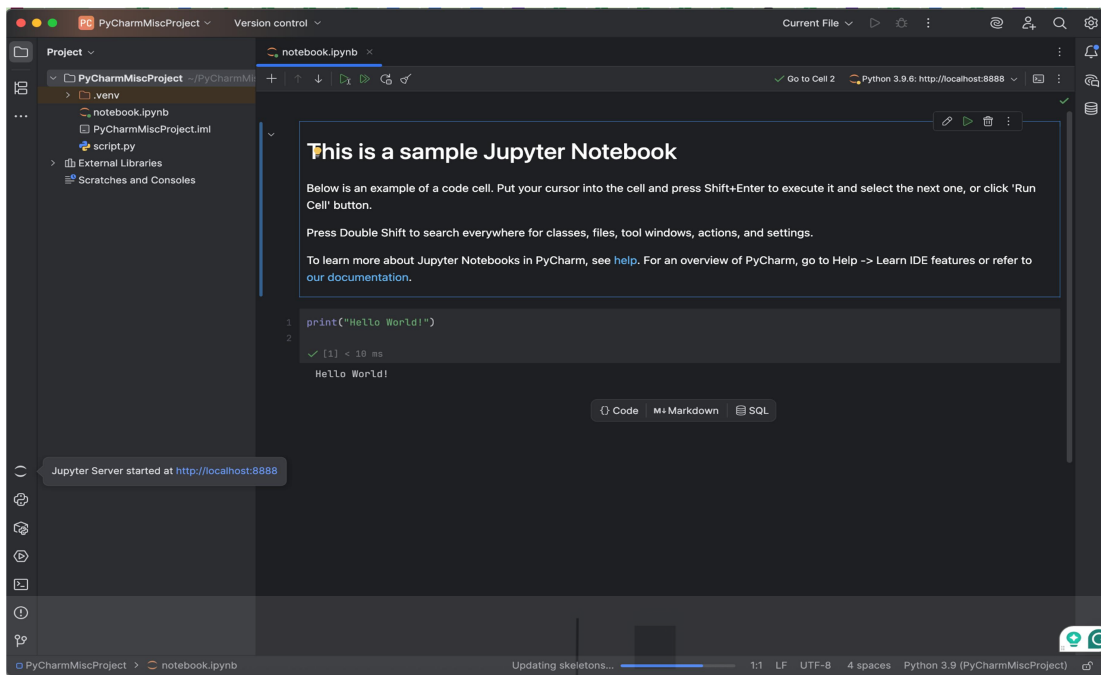


Figure 19: Jupyter Notebook windows with output

Now, let us find the sum of two numbers using Jupyter Notebook and verify the result.

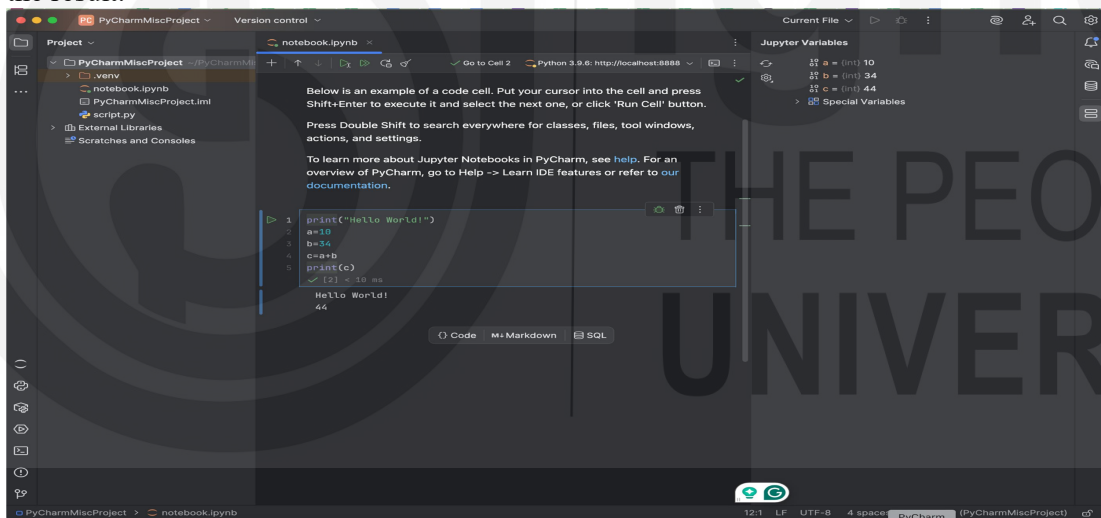


Figure 20: PyCharm Interface of Writing code in Jupyter Notebook with output

Code Block in Jupyter notebook where code is written and executed as shown in the figure 21 below:

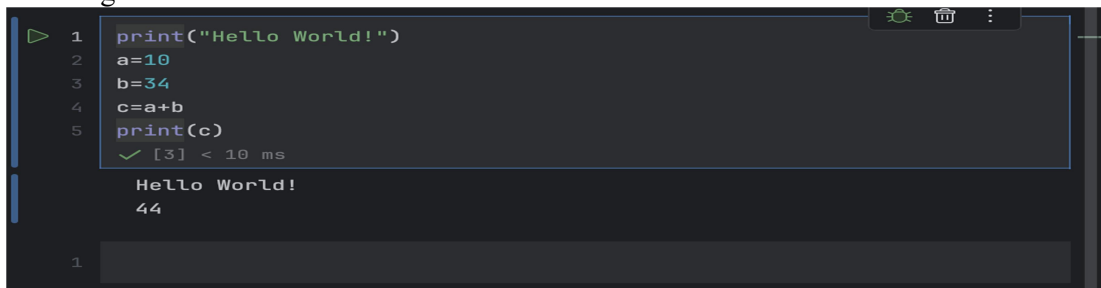


Figure 21: Code Block in Jupyter notebook within PyCharm

Jupyter Variables in Jupyter notebook, where all variables are written in code as shown in the figure 22 below:

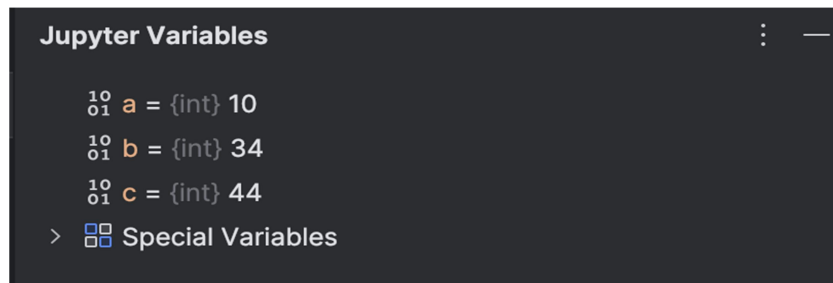


Figure 22: Jupyter Variables Block in PyCharm IDE

Project directory structure in a Jupyter notebook, where all files, folders, configurations, etc., are categorized as shown in the figure 23.

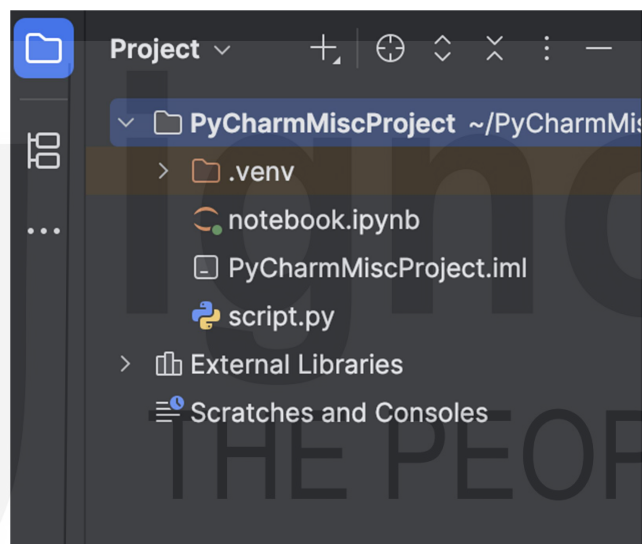


Figure 23: Project directory structure in a Jupyter notebook within PyCharm

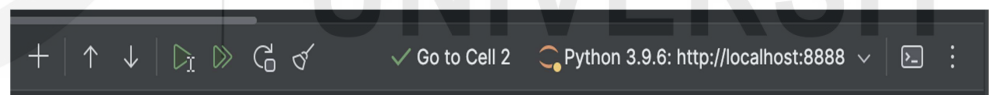


Figure 24: Toolbar in Jupyter notebook in PyCharm

In this tool window, as shown in the above figure, functions are from left to right: Add Code, Move Cell UP, Move Cell Down, Run Cell and select Below, Run All, Restart Kernel, Clear all output, Go to cell, Jupyter Server URL/Configure, Terminal.

Jupyter Server: It can be accessed in a web browser by opening the URL <http://localhost:8888>.

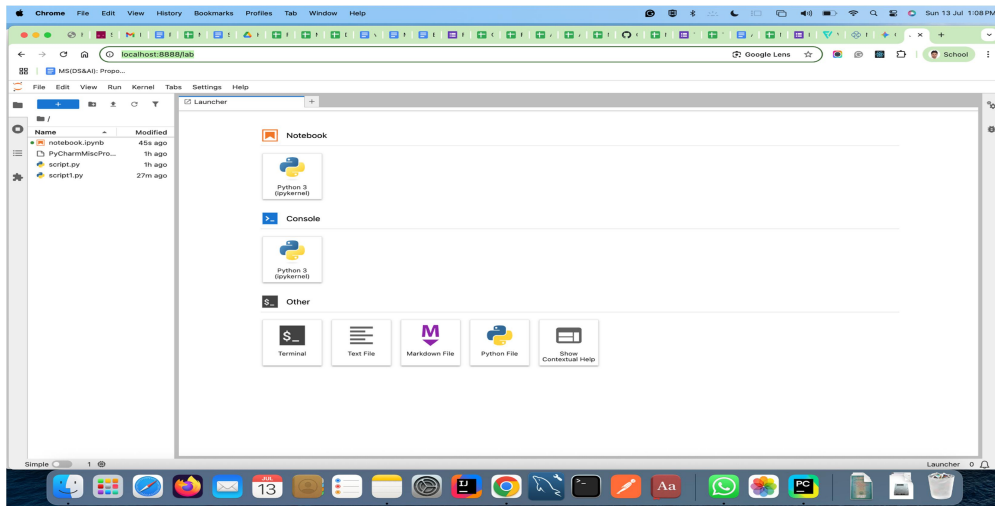


Figure 25: Notebook access in a Web Browser

Some Example Programs for Practice

Example 1: Use of Object, Class and Method in Python

In this example, we will see how to create a class, objects and methods in Python programming.

```
# Initialize the class
class Example1:
    # Class attribute
    total_marks = 60

    # Constructor method to initialize instance attributes
    def __init__(self, name, course, marks_opt):
        self.name = name # Instance attribute
        self.course = course # Instance attribute
        self.marks_opt = marks_opt # Instance attribute

    # Instance method
    def enrolled(self):
        return f"{self.name} has enrolled in \n {self.course} course"

    # Another instance method
    def marks_percentage(self):
        return self.marks_opt / self.total_marks * 100

# Creating an instance (object) of the Example1 class
example = Example1("Vinay Kr Sharma", "Artificial Intelligence", 40)
print(example.enrolled())

# Accessing instance attributes/variable
print(f"Student name is {example.name}.")

## Accessing a class attribute
print(f"Total marks: {example.total_marks}")

# Accessing instance attributes/variable along with function/method
print(f"The Student has scored {example.marks_opt} in the {example.enrolled().}")

# Accessing instance methods
print(f"Average Marks: {example.marks_percentage()}")
```

Output:

```
Vinay Kr Sharma has enrolled in Artificial Intelligence course
Student name is Vinay Kr Sharma.
Total marks: 60
Student has scored 40 in the Vinay Kr Sharma has enrolled in Artificial Intelligence course.
Average Marks: 66.66666666666666
```

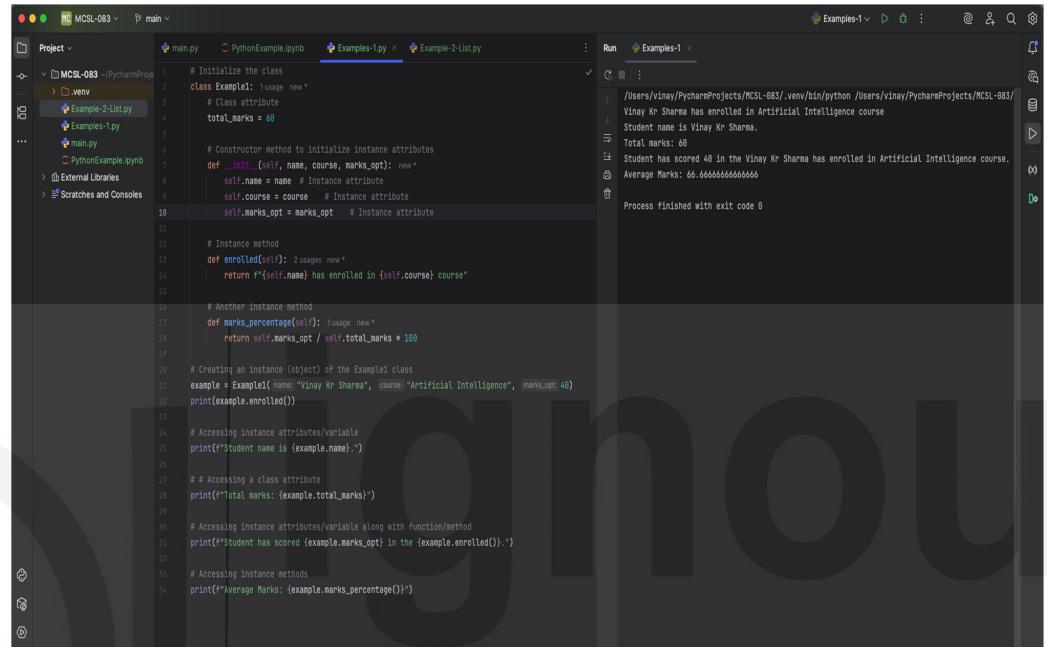


Figure 26: Example 1 in PyCharm IDE

Example 2: Use of List in Python

In this example, we will see how to make use of list along with adding values and iterating over the list variable in Python Programming. List being one of the most essential data structures, practice of its creation and use will help you gain confidence in problem solving.

Steps:

- Creating a list
- Adding values to a list
- Iterating over a list

A. Creating a List

A list is initialized or created using square brackets [].

```
# Initialize the class

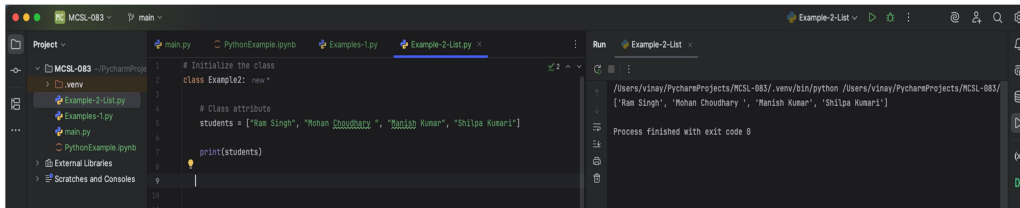
class Example2:

    # Class attribute

    students = ["Ram Singh", "Mohan Choudhary ", "Manish Kumar", "Shilpa Kumari"]
```

```
print(students)
```

Output:



```
# Initialize the class
class Example2:
    # Class attribute
    students = ["Ram Singh", "Mohan Choudhary ", "Manish Kumar", "Shilpa Kumari"]

    print(students)
```

Run Example-2-List

```
/Users/vinay/PycharmProjects/MCSL-083/.venv/bin/python /Users/vinay/PycharmProjects/MCSL-083/
["Ram Singh", 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari']

Process finished with exit code 0
```

B: Adding Values to a List

There are several ways to add elements to a list:

1. using the **append()** function to add at the end of the list value:

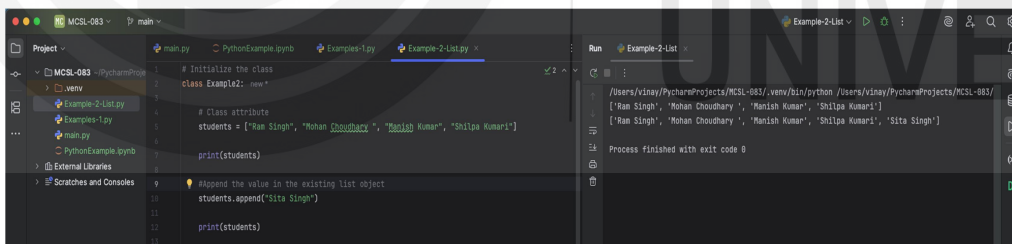
```
# Initialize the class
class Example2:
    # Class attribute
    students = ["Ram Singh", "Mohan Choudhary ", "Manish Kumar", "Shilpa Kumari"]

    print(students)

    #Append the value in the existing list object
    students.append("Sita Singh")

    print(students)
```

Output:



```
# Initialize the class
class Example2:
    # Class attribute
    students = ["Ram Singh", "Mohan Choudhary ", "Manish Kumar", "Shilpa Kumari"]

    print(students)

    #Append the value in the existing list object
    students.append("Sita Singh")

    print(students)
```

Run Example-2-List

```
/Users/vinay/PycharmProjects/MCSL-083/.venv/bin/python /Users/vinay/PycharmProjects/MCSL-083/
["Ram Singh", 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari']
["Ram Singh", 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari', 'Sita Singh']

Process finished with exit code 0
```

2. using the **insert ()** function to adds at a specific position, like **insert (index, value)**

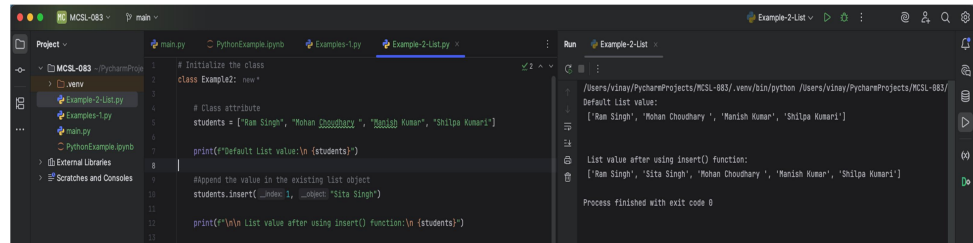
```
# Initialize the class
class Example2:
    # Class attribute
    students = ["Ram Singh", "Mohan Choudhary ", "Manish Kumar", "Shilpa Kumari"]

    print(f"Default List value:\n {students}")

    #Append the value in the existing list object
    students.insert(1, "Sita Singh")

    print(f"\n\n List value after using insert() function:\n {students}")
```

Output:



```
Example-2-List.py
/Users/vinay/PycharmProjects/MCSL-083/.venv/bin/python /Users/vinay/PycharmProjects/MCSL-083/Example-2-List.py
Default List value:
['Ram Singh', 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari']

List value after using insert() function:
['Ram Singh', 'Sita Singh', 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari']

Process finished with exit code 0
```

- Using extend () function to add multiple items at the end of the list value:

```
# Initialize the class
class Example2:

    # Class attribute
    students = ["Ram Singh", "Mohan Choudhary ", "Manish Kumar", "Shilpa Kumari"]

    print(f'Default List value:\n {students}')

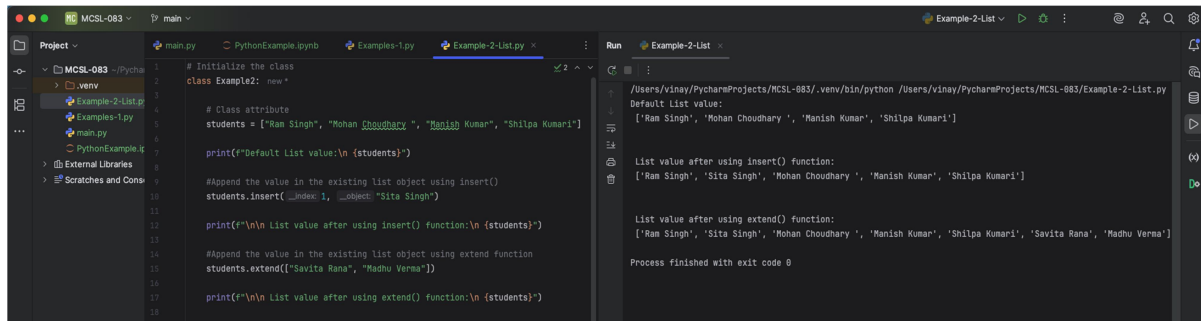
    #Append the value in the existing list object using insert()
    students.insert(1, "Sita Singh")

    print(f'\n\n List value after using insert() function:\n {students}')

    #Append the value in the existing list object using the extend function
    students.extend(["Savita Rana", "Madhu Verma"])

    print(f'\n\n List value after using extend() function:\n {students}')
```

Output:



```
Example-2-List.py
/Users/vinay/PycharmProjects/MCSL-083/.venv/bin/python /Users/vinay/PycharmProjects/MCSL-083/Example-2-List.py
Default List value:
['Ram Singh', 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari']

List value after using insert() function:
['Ram Singh', 'Sita Singh', 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari']

List value after using extend() function:
['Ram Singh', 'Sita Singh', 'Mohan Choudhary ', 'Manish Kumar', 'Shilpa Kumari', 'Savita Rana', 'Madhu Verma']

Process finished with exit code 0
```

C. Iterating Over a List

- Iterate the List using a for loop

```
# Iterate the List using a for loop in-line syntax
```

```
for student in students: print(student)
```

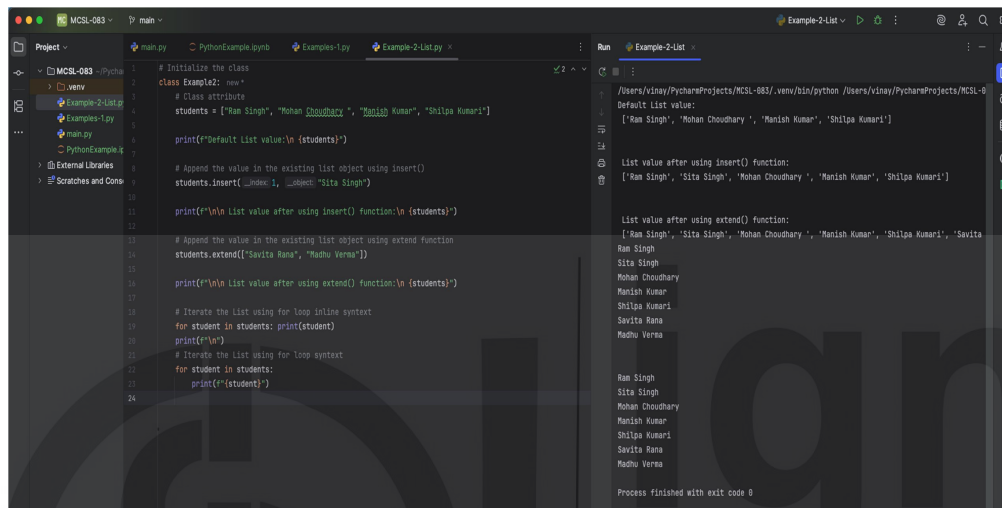
```
print(f"\n")
```

```
# Iterate the List using for loop syntax
```

```
for student in students:
```

```
    print(f"{student}")
```

Output:



```
# Initialize the class
class Example2:
    # Class attribute
    students = ["Ran Singh", "Mohan Choudhary", "Manish Kumar", "Shilpa Kumari"]

    print(f"Default List value:\n {students}")

    # Append the value in the existing list object using insert()
    students.insert(1, "Sita Singh")

    print(f"\n\n List value after using insert() function:\n {students}")

    # Append the value in the existing list object using extend function
    students.extend(["Savita Rana", "Madhu Verma"])

    print(f"\n\n List value after using extend() function:\n {students}")

    # Iterate the List using for loop inline syntax
    for student in students: print(student)
    print(f"\n")

    # Iterate the List using for loop syntax
    for student in students:
        print(f"{student}")
```

Output:

```
/Users/vinay/PycharmProjects/MCSL-083/.venv/bin/python /Users/vinay/PycharmProjects/MCSL-083/Example-2-List.py
Default List value:
['Ran Singh', 'Mohan Choudhary', 'Manish Kumar', 'Shilpa Kumari']

List value after using insert() function:
['Ran Singh', 'Sita Singh', 'Mohan Choudhary', 'Manish Kumar', 'Shilpa Kumari']

List value after using extend() function:
['Ran Singh', 'Sita Singh', 'Mohan Choudhary', 'Manish Kumar', 'Shilpa Kumari', 'Savita Rana', 'Madhu Verma']

Ran Singh
Sita Singh
Mohan Choudhary
Manish Kumar
Shilpa Kumari
Savita Rana
Madhu Verma

Ran Singh
Sita Singh
Mohan Choudhary
Manish Kumar
Shilpa Kumari
Savita Rana
Madhu Verma

Process finished with exit code 0
```

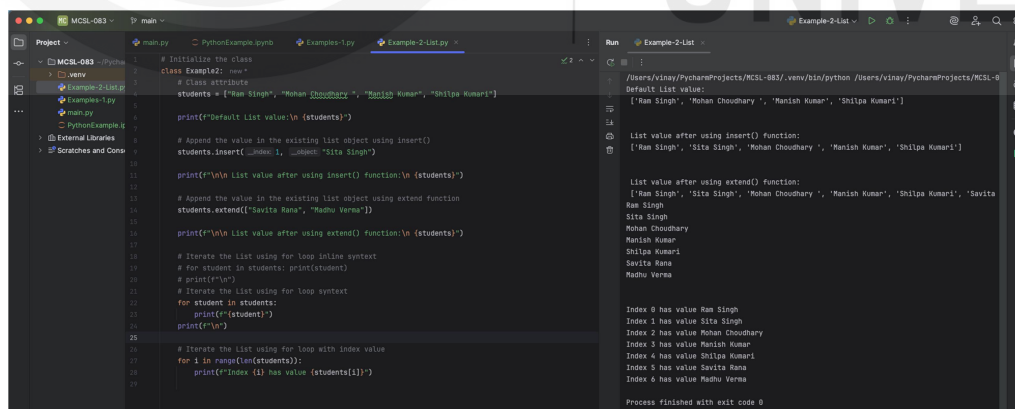
2. Using a for loop with range () and len ()

```
# Iterate the List using for loop with an index value
```

```
for i in range(len(students)):
```

```
    print(f"Index {i} has value {students[i]}")
```

Output:



```
# Initialize the class
class Example2:
    # Class attribute
    students = ["Ran Singh", "Mohan Choudhary", "Manish Kumar", "Shilpa Kumari"]

    print(f"Default List value:\n {students}")

    # Append the value in the existing list object using insert()
    students.insert(1, "Sita Singh")

    print(f"\n\n List value after using insert() function:\n {students}")

    # Append the value in the existing list object using extend function
    students.extend(["Savita Rana", "Madhu Verma"])

    print(f"\n\n List value after using extend() function:\n {students}")

    # Iterate the List using for loop inline syntax
    for student in students: print(student)
    print(f"\n")

    # Iterate the List using for loop syntax
    for student in students:
        print(f"{student}")

    # Iterate the List using for loop with index value
    for i in range(len(students)):
        print(f"Index {i} has value {students[i]}")
```

Output:

```
/Users/vinay/PycharmProjects/MCSL-083/.venv/bin/python /Users/vinay/PycharmProjects/MCSL-083/Example-2-List.py
Default List value:
['Ran Singh', 'Mohan Choudhary', 'Manish Kumar', 'Shilpa Kumari']

List value after using insert() function:
['Ran Singh', 'Sita Singh', 'Mohan Choudhary', 'Manish Kumar', 'Shilpa Kumari']

List value after using extend() function:
['Ran Singh', 'Sita Singh', 'Mohan Choudhary', 'Manish Kumar', 'Shilpa Kumari', 'Savita Rana', 'Madhu Verma']

Ran Singh
Sita Singh
Mohan Choudhary
Manish Kumar
Shilpa Kumari
Savita Rana
Madhu Verma

Index 0 has value Ran Singh
Index 1 has value Sita Singh
Index 2 has value Mohan Choudhary
Index 3 has value Manish Kumar
Index 4 has value Shilpa Kumari
Index 5 has value Savita Rana
Index 6 has value Madhu Verma

Process finished with exit code 0
```

A complete example of Use of a list:

```
# Initialize the class
```

```
class Example2:
```

```
    # initialise empty list attribute
```

```
    students = []
```

```

#add value using append() value one by one
students.append("Ram Singh")
students.append("Mohan Choudhary")
students.append("Manish Kumar")
students.append("Shilpa Kumari")

# add value using insert() value at 3rd index
students.insert(3, "Sita Singh")

# add multiple values using extend()
students.extend(["Savita Rana", "Madhu Verma"])

# Print default string
print("List of students:")

# Iterate the list using for loop
for student in students:
    print(f'{student}')

```

Output:

```

class Example2:
    def __init__(self):
        self.students = []

    def add_value(self, value):
        self.students.append(value)

    def add_value_at_index(self, index, value):
        self.students.insert(index, value)

    def add_multiple_values(self, values):
        self.students.extend(values)

    def print_students(self):
        print("List of students:")
        for student in self.students:
            print(student)

# Create an instance of Example2
example2 = Example2()

# Add values
example2.add_value("Ram Singh")
example2.add_value("Mohan Choudhary")
example2.add_value("Manish Kumar")
example2.add_value("Shilpa Kumari")

# Add value at index
example2.add_value_at_index(3, "Sita Singh")

# Add multiple values
example2.add_multiple_values(["Savita Rana", "Madhu Verma"])

# Print students
example2.print_students()

```

Output:

```

List of students:
Ram Singh
Mohan Choudhary
Manish Kumar
Sita Singh
Shilpa Kumari
Savita Rana
Madhu Verma

```

Example 3: Use of Set in Python

In this example, we will see how use of set including h adding values and iterating over the set variable in Python Programming.

As you know a **set** is an **unordered**, **unindexed** collection of **unique** items. It is useful when:

1. You want to store only unique elements (no duplicates).
2. You do not care about the order.

Steps:

1. Creating a set

```
students = set()
```

2. Adding values to a set

```
3. students.add("Ram Singh")
4. students.add("Shilpa Kumari")
5. students.add("Mohan Choudhary")
6. students.add("Manish Kumar")
```

3. Update multiple values

```
# add multiple value using extend()

students.update(["Savita Rana", "Madhu Verma"])
```

4. Iterating over a set

```
# Iterate the set using a for loop

for student in students:
    print(f"{student}")
```

Note: The output order might vary each time you run it. Think why?

A complete Example of Use of a set:

```
# Initialize the class
class Example2:
    # initialise empty list attribute
    students = set()

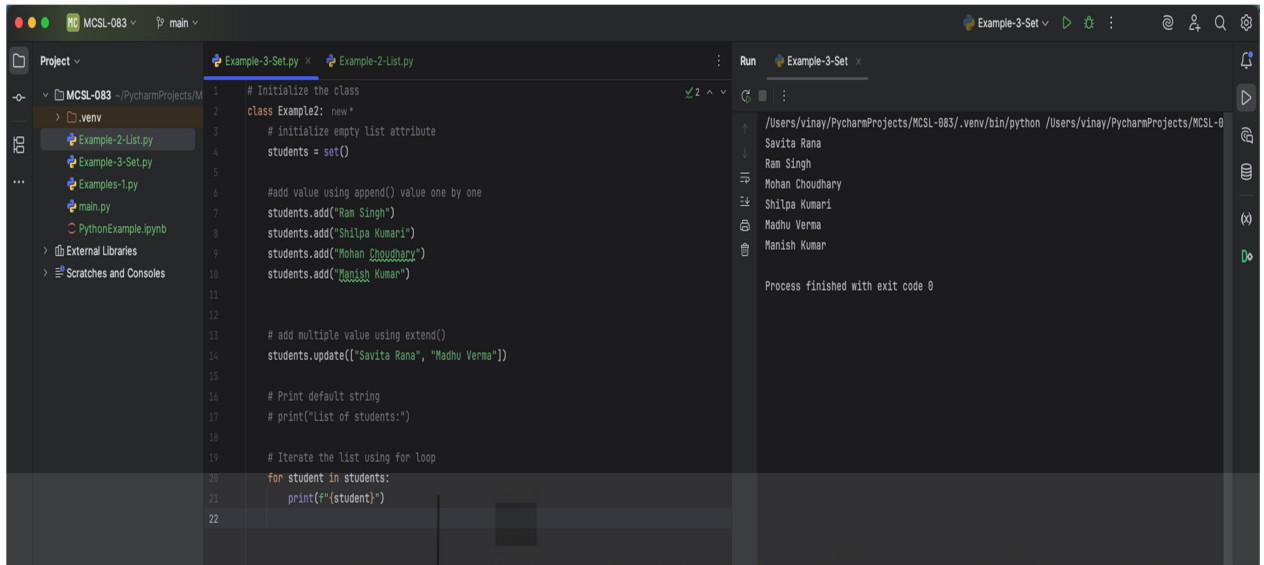
    #add value using append() value one by one
    students.add("Ram Singh")
    students.add("Shilpa Kumari")
    students.add("Mohan Choudhary")
    students.add("Manish Kumar")

    # add multiple values using extend()
    students.update(["Savita Rana", "Madhu Verma"])

    # Print default string
    # print("List of students:")

    # Iterate the list using for loop
    for student in students:
        print(f"{student}")
```

Output:



The screenshot shows a code editor with a project named 'MCSL-083'. The file 'Example-3-Set.py' is open, displaying a Python class 'Example2'. The code initializes a class, creates an empty list 'students', and adds several student names using 'append()' and 'extend()'. The output window on the right shows the names of the students: Savita Rana, Ram Singh, Mohan Choudhary, Shilpa Kumari, Madhu Verma, and Manish Kumar. The process finished with exit code 0.

```
1 # Initialize the class
2 class Example2: new *
3     # initialize empty list attribute
4     students = set()
5
6     #add value using append() value one by one
7     students.add("Ram Singh")
8     students.add("Shilpa Kumari")
9     students.add("Mohan Choudhary")
10    students.add("Manish Kumar")
11
12
13    # add multiple value using extend()
14    students.update(["Savita Rana", "Madhu Verma"])
15
16    # Print default string
17    # print('List of students:')
18
19    # Iterate the list using for loop
20    for student in students:
21        print(f'{student}')
22
```

Output:

```
/Users/vinay/PycharmProjects/MCSL-083/.venv/bin/python /Users/vinay/PycharmProjects/MCSL-0
Savita Rana
Ram Singh
Mohan Choudhary
Shilpa Kumari
Madhu Verma
Manish Kumar

Process finished with exit code 0
```

Example 4: Connection to Database in Python

In this example, we will see how to set up a database connection with MySQL and display the value from a table using Python programming. To perform this task, you need to install the **MySQL Database, mysql-connector-python** package provided by Oracle. If you want to access the database using GUI, you must install MySQL-Workbench, which provides you with user-friendly interface to manage databases. Please refer to the official Installation guide by visiting the following URL:

<https://dev.mysql.com/doc/refman/8.4/en/installing.html>. Here we have used **MySQL 8.4**, you can choose as per your requirement and interest.

To set up a database connection with MySQL and display values from a table using Python, you can use the popular **mysql-connector-python** package provided by Oracle.

Step 1: Prepare the database and table

For this example, student table has been created using the following command:

```
CREATE TABLE `python`.`student` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `name` VARCHAR (45) NULL,
  `enrollment` VARCHAR (45) NULL,
  `email_id` VARCHAR (45) NULL,
  `course` VARCHAR (45) NULL,
  PRIMARY KEY (`id`));
```

Step 2: Install MySQL Connector:

Run this command in your terminal or command prompt:

pip install mysql-connector-python

Step 3: Import the Module and connect to MySQL:

```
import mysql.connector

try:
    # Establish connection
    conn = mysql.connector.connect(
        host="localhost",
        user="root",
        password="Sau@1234",
        database="python"
    )
    # Create a cursor object
    cursor = conn.cursor()
```

Step 4: Fetch and Display Data from a student table:

```
# SQL query to fetch all rows
cursor.execute("SELECT * FROM student")

# Fetch and display data
rows = cursor.fetchall()
print("Student Records:")
for row in rows:
    print(f"ID: {row[0]}, Name: {row[1]}, Enrollment: {row[2]}, Course: {row[4]}")
```

Step 5: Close the connection

```
finally:
    # Always close the cursor and the connection
    if cursor:
        cursor.close()
    if conn:
        conn.close()
```

Complete Python Program:

```
import mysql.connector

try:
    # Establish connection
    conn = mysql.connector.connect(
        host="localhost",
        user="root",
        password="Sau@1234",
        database="python"
    )
    # Create a cursor object
    cursor = conn.cursor()
```

```
# SQL query to fetch all rows

cursor.execute("SELECT * FROM student")

# Fetch and display data

rows = cursor.fetchall()

print("Student Records:")

for row in rows:

    print(f'ID: {row[0]}, Name: {row[1]}, Enrollment: {row[2]}, Course: {row[4]}')

except mysql.connector.Error as err:

    print("Error:", err)

finally:

    # Always close the cursor and the connection

    if cursor:

        cursor.close()

    if conn:

        conn.close()
```

The screenshot displays the PyCharm IDE interface. The main editor window shows a Python script that connects to a MySQL database using the mysql.connector module. The script executes a SELECT query to fetch all rows from a table named 'student'. The fetched data is then printed in a formatted string. The script includes error handling for MySQL connector errors and a finally block to ensure the cursor and connection are properly closed.

The terminal at the bottom shows the execution of the command `pip install mysql-connector-python`. The output indicates that the package was successfully installed, along with its dependencies. The terminal also shows the output of the script, which prints the student records.

Figure 27: Screenshot of running example 5

Explanation of the screenshot:

- The bottom of the IDE is the terminal where the connector command is executed to install MySQL-connector for Python

- The left-hand side of the IDE is the directory structure
- The middle part of the IDE source code is written
- The right side of the IDE is the output of the example, where we fetch the student information from the database.

Example 5: Creation Array using NumPy

In this example, we will see how to create an array and display it using NumPy library.

Let us see how to **create an array** and **display it** using **NumPy** (it is a powerful library for numerical operations in Python).

Step 1: Before using NumPy, you have to install the NumPy Library using following command in terminal:

`pip install NumPy`



Step 2: Once successfully installed, create a class and import NumPy library within class where we are using it, using the following command:

```
import numpy as np
```

Step 3: Now create NumPy Array

```
# Create arrays

arr1 = np.array([10, 20, 30, 40, 50]) # 1D array

arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array (matrix)

arr3 = np.array(["Ram Singh", "Mohan Choudhary", "Sita Singh"]) # String array
```

Step 4: Display the Array

```
# Display arrays

print("1D Array:")

print(arr1)

print("\n2D Array:")

print(arr2)

print("\nString Array:")

print(arr3)
```

Complete Example of Creating and Using Array:

```
import numpy as np

class Example4NumPy:

# Create arrays

arr1 = np.array([10, 20, 30, 40, 50]) # 1D array

arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array (matrix)

arr3 = np.array(["Ram Singh", "Mohan Choudhary", "Sita Singh"]) # String array

# Display arrays

print("1D Array:")

print(arr1)

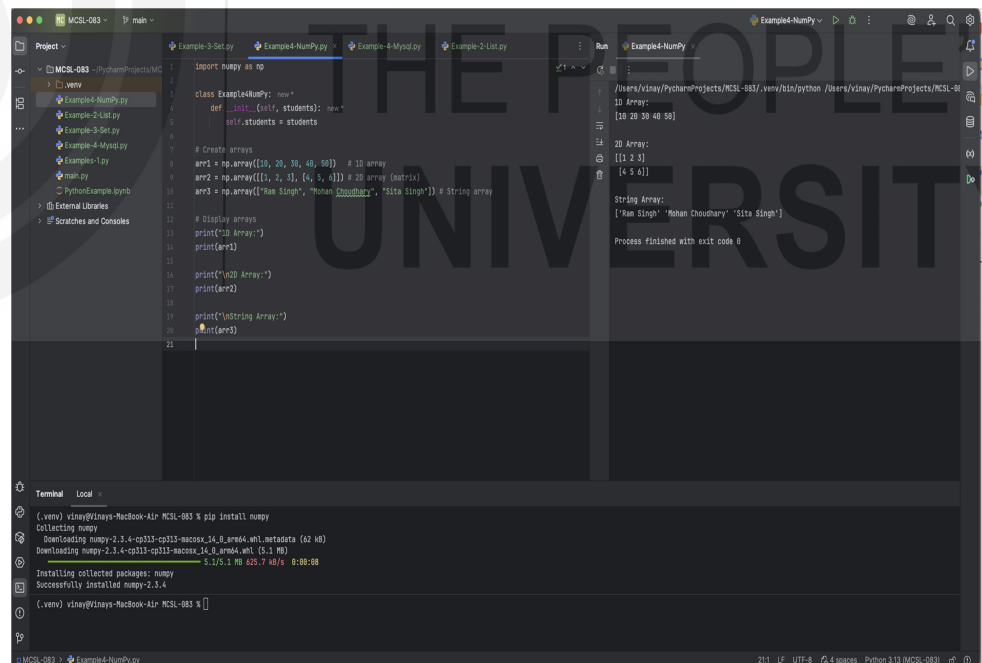
print("\n2D Array:")

print(arr2)

print("\nString Array:")

print(arr3)
```

Output:



```
import numpy as np

class Example4NumPy:

# Create arrays

arr1 = np.array([10, 20, 30, 40, 50]) # 1D array

arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array (matrix)

arr3 = np.array(["Ram Singh", "Mohan Choudhary", "Sita Singh"]) # String array

# Display arrays

print("1D Array:")

print(arr1)

print("\n2D Array:")

print(arr2)

print("\nString Array:")

print(arr3)
```

Output:

```
1D Array:
[10 20 30 40 50]

2D Array:
[[1 2 3]
 [4 5 6]]

String Array:
['Ram Singh' 'Mohan Choudhary' 'Sita Singh']

Process finished with exit code 0
```

You are advised to refer to the NumPy Manual for more useful NumPy Array Functions: <https://numpy.org/devdocs/reference/generated/numpy.array.html>

Example 6: Plot a chart using PyPlot in Python

Here, we will create a GUI interface using PyQt / PySide. To plot a chart using pyplot, we need to use the matplotlib library — specifically the pyplot module, which provides a MATLAB-like interface for plotting in Python.

Step by step procedure to create GUI

Step 1: Use the following command to Install the *matplotlib* using terminal or command prompt:

pip install matplotlib

```
Terminal Local v + v
(.venv) vinnay@Innys-MacBook-Air: MCL-083 % pip install matplotlib
Collecting matplotlib
  Downloading matplotlib-3.10.7-cp311-cp311-macosx_11_0_arm64.whl.metadata (11 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.0-cp311-cp311-macosx_11_0_arm64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (5.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.48.1-cp311-cp311-macosx_10_11_universal2.whl.metadata (112 kB)
Collecting kiwisolver>=3.1 (from matplotlib)
  Downloading kiwisolver-1.4.9-cp311-cp311-macosx_11_0_arm64.whl.metadata (6.3 kB)
Requirement already satisfied: numpy>=1.23.0 in ./venv/lib/python3.11/site-packages (from matplotlib) (2.1.4)
Collecting packaging>=20.0 (from matplotlib)
  Using cached packaging-25.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pillow>=8 (from matplotlib)
  Downloading pillow-12.0.0-cp311-cp311-macosx_11_0_arm64.whl.metadata (8.8 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.2.1-py3-none-any.whl.metadata (5.0 kB)
Collecting python-dateutil>=2.7 (from matplotlib)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.7->matplotlib)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading matplotlib-3.10.7-cp311-cp311-macosx_11_0_arm64.whl (8.1 MB)
8.1/8.1 MB 499.6 kB/s 0:00:16
⚙️ Downloading contourpy-1.3.0-cp311-cp311-macosx_11_0_arm64.whl (274 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
⚙️ Downloading fonttools-4.48.1-cp311-cp311-macosx_10_11_universal2.whl (2.8 MB)
2.8/2.8 MB 761.2 kB/s 0:00:03
⚙️ Downloading kiwisolver-1.4.9-cp311-cp311-macosx_11_0_arm64.whl (64 kB)
Using cached packaging-25.0-py3-none-any.whl (64 kB)
⚙️ Downloading pillow-12.0.0-cp311-cp311-macosx_11_0_arm64.whl (4.7 MB)
4.7/4.7 MB 695.0 kB/s 0:00:06
⚙️ Downloading pyparsing-3.2.1-py3-none-any.whl (113 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, pyparsing, pillow, packaging, kiwisolver, fonttools, cycler, contourpy, python-dateutil, matplotlib
Successfully installed contourpy-1.3.0 cycler-0.12.1 fonttools-4.48.1 kiwisolver-1.4.9 matplotlib-3.10.7 packaging-25.0 pillow-12.0.0 pyparsing-3.2.1 python-dateutil-2.9.0.post0 six-1.17.0
(.venv) vinnay@Innys-MacBook-Air: MCL-083 %
```

Step 2: Import pyplot using following syntax:

```
import matplotlib.pyplot as plt
```

Step 3: Initialize the data for X-axis and Y-axis using the following syntax for different type of charts:

```
# Initialize X-axis and Y-axis for Basic Line Chart
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]
```

```
# Initialize X-axis and Y-axis for Bar Chart
x1 = ["Ram", "Mohan", "Sita"]
y1 = [95, 77, 83]
```

```
# Initialize X-axis and Y-axis for Pie Chart
labels = ["Python", "Java", "C++"]
sizes = [40, 35, 25]
```

```
# Initialize X-axis and Y-axis for Scatter Plot
```

```
x = [1, 2, 3, 4, 5]
y = [10, 12, 25, 30, 22]
```

Step 4: Plotting Chart using following syntax for different type of charts:

```
# Plotting Basic Line Chart

plt.plot(x, y, color='green', marker='o', linestyle='--')

plt.title("Growth Over Time")

plt.xlabel("Time")

plt.ylabel("Value")

plt.grid(True)

plt.show()
```

```
# Plotting Bar Chart

plt.bar(x1, y1, color='red')

plt.title("Grade Chart Example")

plt.xlabel("Student Name")

plt.ylabel("Marks")

plt.show()
```

```
# Plotting Pie Chart

plt.pie(sizes, labels=labels, autopct="%1.1f%%")

plt.title("Programming Language Usage")

plt.show()
```

```
# Plotting Scatter Plot

plt.scatter(x, y, color='red', marker='o', linestyle='--')

plt.title("Scatter Plot Example")
```

A complete example code for GUI creation:

```
import matplotlib.pyplot as plt

class Example5GUI:

    # Initialize X-axis and Y-axis for Basic Line Chart

    x = [1, 2, 3, 4, 5]

    y = [10, 20, 25, 30, 40]

    # Plotting Basic Line Chart

    plt.plot(x, y, color='green', marker='o', linestyle='--')

    plt.title("Growth Over Time")
```

```

plt.xlabel("Time")
plt.ylabel("Value")
plt.grid(True)
plt.show()

# Initialize X-axis and Y-axis for Bar Chart
x1 = ["Ram", "Mohan", "Sita"]
y1 = [95, 77, 83]

# Plotting Bar Chart
plt.bar(x1, y1, color='red')

plt.title("Grade Chart Example")
plt.xlabel("Student Name")
plt.ylabel("Marks")
plt.show()

# Initialize X-axis and Y-axis for Pie Chart
labels = ["Python", "Java", "C++"]
sizes = [40, 35, 25]

# Plotting Pie Chart
plt.pie(sizes, labels=labels, autopct="%1.1f%%")
plt.title("Programming Language Usage")
plt.show()

# Initialize X-axis and Y-axis for Scatter Plot
x = [1, 2, 3, 4, 5]
y = [10, 12, 25, 30, 22]

# Plotting Scatter Plot
plt.scatter(x, y, color='red', marker='o', linestyle='--')
plt.title("Scatter Plot Example")
plt.show()

```

Some Important Tips:

- Always call **plt.show()** to display the chart.
- Use **marker, color, linestyle** to customize appearance.
- Use **plt.grid(True)** to add grid lines.

Output:

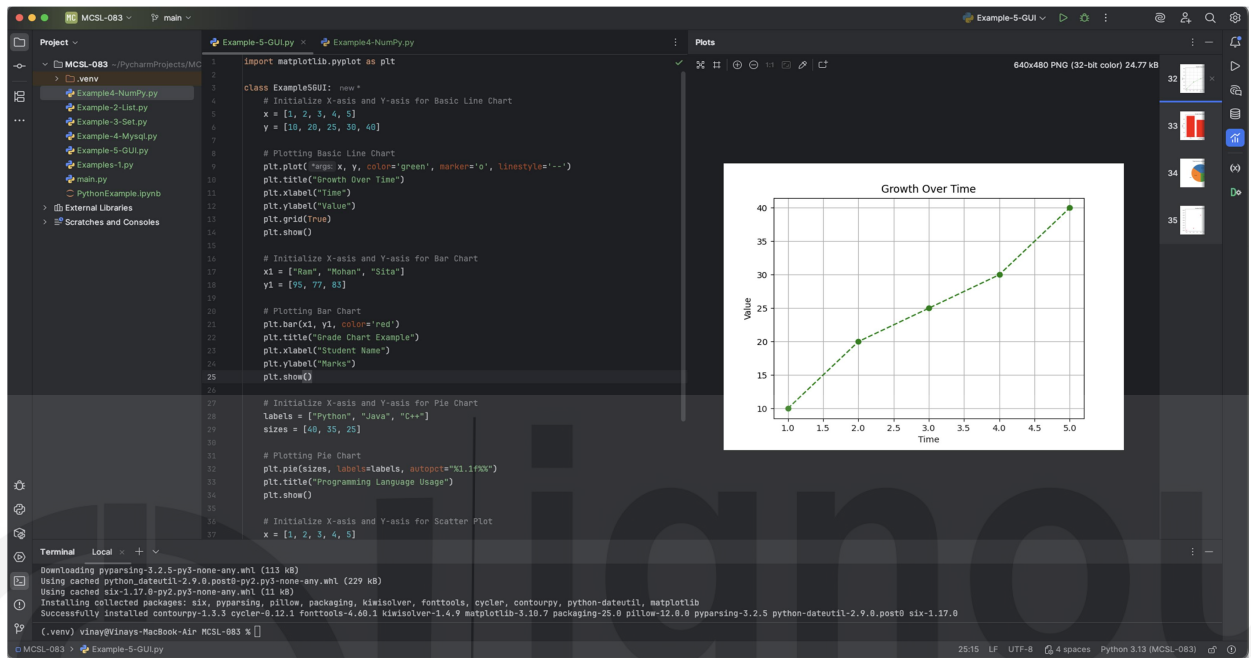


Figure 28: Output of the Example of Plotting a Basic Line Chart in IDE

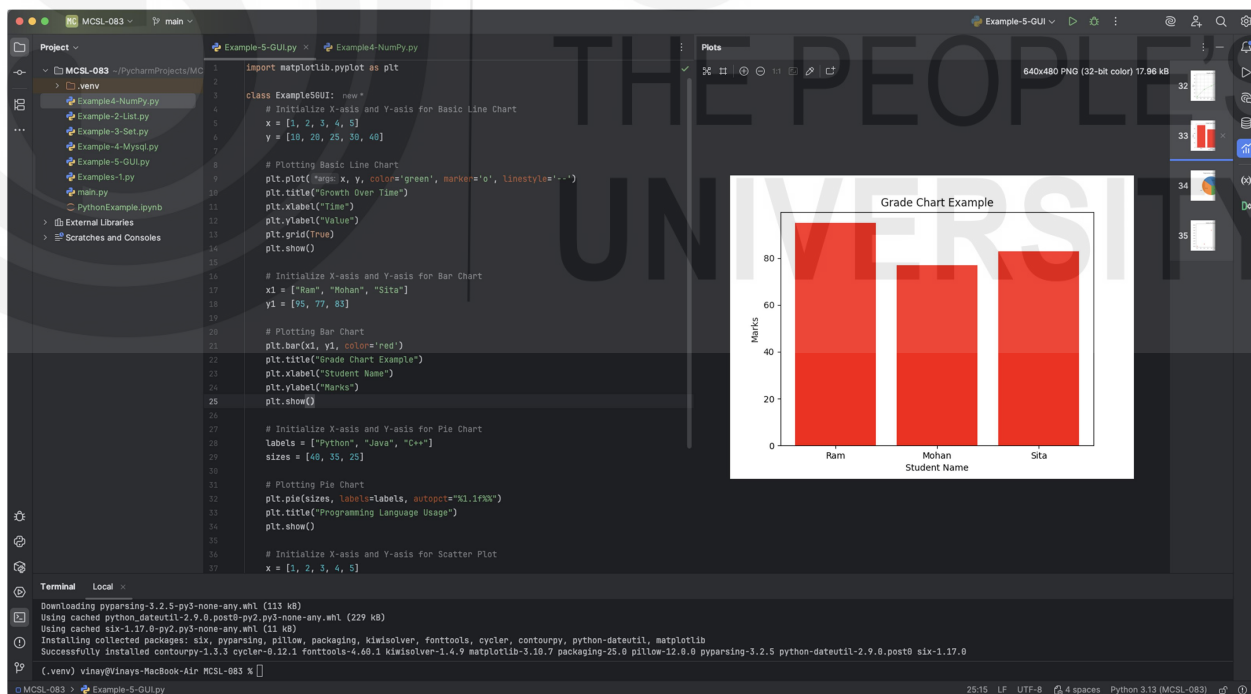
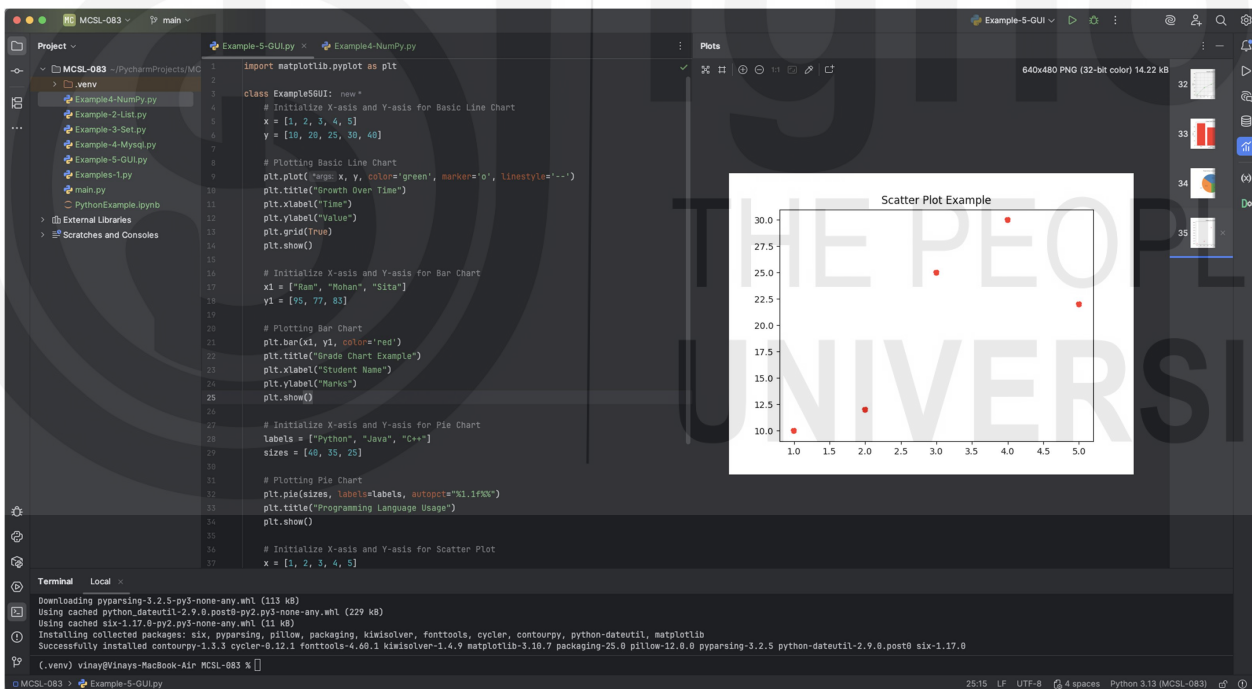
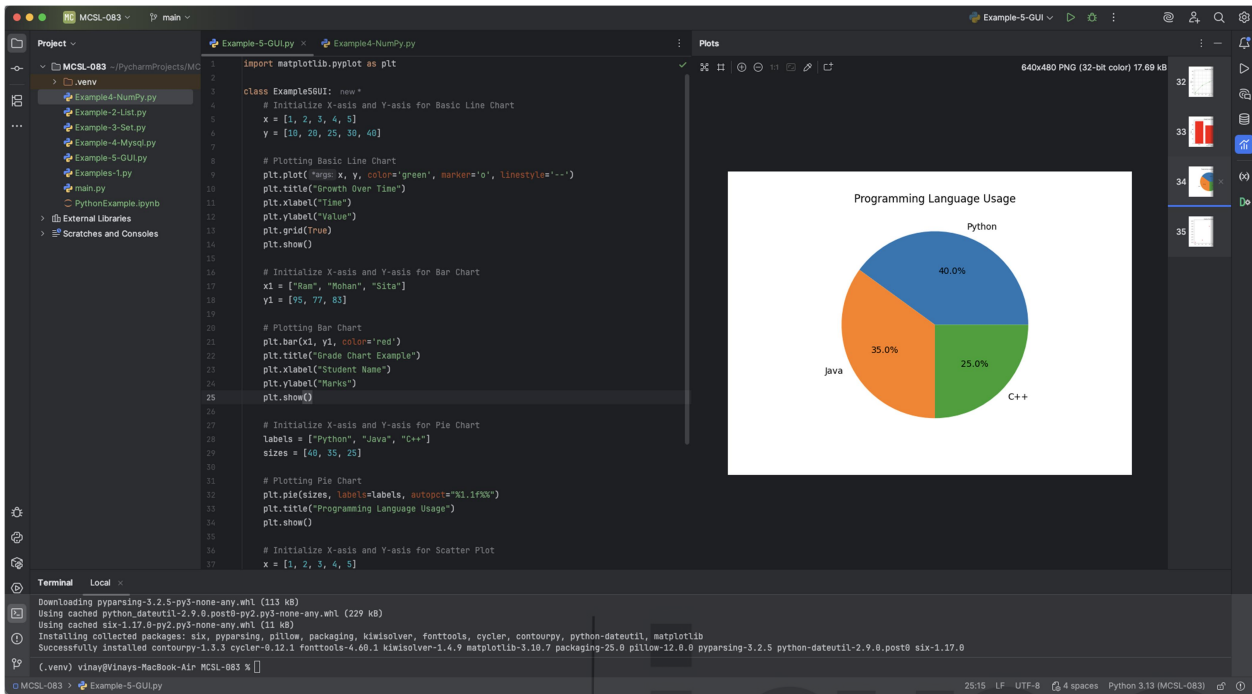


Figure 29: Output of the Example of Plotting a Bar Chart in the IDE



1.4 LIST OF LAB ASSIGNMENTS (SESSION WISE)

SESSION-1(Based on MCS-081: Unit 2 and Unit 3)

1. Write a program to find the transpose of the following matrix:

$$A = \begin{bmatrix} 80 & 70 \\ 36 & 26 \end{bmatrix}$$

2. Create a singly linked list class and implement insert at beginning, insert at end, delete by value, and traverse operations.
3. Create a list with integer values and write a recursive function to perform the merge sort on the same list. Track and print the number of recursive calls.
4. Write a Python program to create a tuple of at least 10 integers and perform the following:
 - Find the maximum, minimum, and sum of elements
 - Access elements using positive and negative indexing
 - Count the occurrence of a specific element
5. Write a Python program to conversation between Tuple and List and perform the following conversation:
 - Convert a tuple into a list
 - Modify the list (e.g., add or remove an element)
 - Convert it back to a tuple
6. Create a dictionary of 5 Employees with their EMP_ID as keys and Salary as values. Write code to:
 - Add a new Employee
 - Update Salary for an existing Employee
 - Delete an Employee
 - Find the Employee with the 2nd highest Salary in the dictionary.
7. Given two dictionaries, merge them. If a key is present in both, sum their values.
8. Write a Python program that take a string as input and find the number of digits, uppercase letters and lowercase letters.
9. Write a Python program which tasked the age of a person as input and print a message whether the person is eligible to vote or not.
10. Write a Python program which take a score between 1 and 10 as input. If the score is out of range, print an error. If the score is between 1 and 10, print a grade as per the following table:

Score	Grade
>= 9	A+
>= 8	A
>= 7	B+
>= 6	B
>= 5	C+
>= 4	C
< 4	Fail

SESSION-2 (Based on MCS-081: Unit 4)

1. Write a Python function **is prime(n)** that returns **true** if the number **is prime**, and **false otherwise**.
2. Write a recursive function to calculate the factorial of a number. Test with inputs like 5, 7, and 10 etc.
3. Create a function **count vowel(s)** that takes a string as input and returns the number of vowels in it.

Test it with: "Hello, Welcome to Python Lab Section"

4. Write a program to read the file **data.txt** and display its contents line by line with line numbers.
5. Design a function **validate password (password)** that checks whether a password is strong or not. Note that the password must be at least 8 characters long, contain at least one uppercase letter, one lowercase letter, one number, and one special character.
6. Define a recursive function to traverse a nested dictionary and print all keys and their corresponding values.
7. Write a Python program that copies contents from one file (**source.txt**) to another file (**destination.txt**).
8. Create a Python script to read a CSV file named **students.csv** containing student records (Name, Roll, Marks).

Write functions to:

- a. Read the CSV file into a list of dictionaries
 - b. Calculate average marks
 - c. Write students who scored above average into **top_students.csv**.
9. Define a recursive function to traverse a nested dictionary and print all keys and their corresponding values.
Read the nested dictionary from a JSON file (config.json).
 10. Write a Python program that includes a function **log event(message, level)** to log events into a file **log.txt**. Each log entry should include a **timestamp**, **log level (INFO, ERROR, DEBUG)**, and the message. Implement proper time-stamping using the datetime module.

SESSION-3 Based on MCS-081: Unit-5)

1. Define three Python classes: Student, Courses and Marks using the inheritance concept with the following variables:
 - Students: ID, Roll number, name, Email ID, mobile no.
 - Courses: ID, Code, title
 - Marks: Student id, Course id, Max Marks, Min marks, marks obtained

Create a Result class having the method which show the first semester results in which students have opted for 5 courses. Also define following methods in the class:

- calculate average (),
 - get grade () (based on average),
 - display result ()
2. Create a Python class Calculator with followings:
- Methods: add (x, y), subtract(x, y), multiply(x, y), divide(x, y).

Write a menu-driven program to use these methods.

SESSION-4 Based on MCS-081: Unit-6 and Unit 7)

1. Write Python program for Exception Handling using the following concepts:
- a. try-except
 - b. try-except-else-finally

Test the following exceptions in your program:

- List Index Out of Range
 - File not found exception
 - Divided by Zero
 - Index out of bounds Exception
 - InvalidAgeException
 - TypeError
 - AttributeError
 - ValueError
 - Custom Exception: Age Validation
2. Write a programme to create a Python module named **mycalculator.py** that contains functions for **add, subtract, multiply, and divide**. Then, create another script to import and use these functions.
3. Write a program to import only the add function from the **mycalculator.py** module (above) using **from mycalculator import add** and demonstrate its use.
4. Write a program to create a package named geometry with the following structure:

```

geometry/
|
|— __init__.py
|— circle.py # Contains area and circumference of a circle
|— square.py # Contains area and perimeter of a Square
|— rectangle.py # Contains area and perimeter of a rectangle

```

Then write a program that imports and uses these modules.

SESSION-5 (Based on MCS-081: Unit 8 and Unit-9)

Unit-8: Thread Handling in Python

1. Write a Python code to create a thread that prints numbers from 1 to 100 with a 10-second delay.
2. Create multiple threads for executing multiple processes and print them, for example one thread for addition, one thread for subtraction, one thread for multiplication and one thread for division.
3. Write a program to demonstrate race condition and prevent it by using a lock.
4. Write a program to create a daemon thread that runs in the background of the application.
5. Write a program to run multiple countdown timers in parallel.
6. Design the Login Windows with fields username, password and button submit. Display the welcome message after successfully login.
7. Design a simple To-Do List application with following function:
 - a. An Entry field to type tasks
 - b. An "Add Task" button
 - c. A listbox that will display all added tasks and mark completed or pending as current status.

SESSION-6 (Based on MCS-081: Unit-10)

1. Create the RESTful APIs in Python to display the Student Result.
2. Modify the code to return appropriate HTTP status codes for different operations like success, not found, null, bad request, etc.
3. Modify the API to allow access of API securely.
4. Modify the GET endpoints to support pagination using query parameters.

SESSION-7 Based on MCS-081: Unit-11 and Unit 12)

1. Create a Python program to read, write and display data from various files like .csv, .json, and .txt.
2. Write a Python program to perform the CRUD operation from the database to show the student's result, using any database as per your choice.
3. Write a Python program to read student results from the database and export in a .pdf format.
4. Write a Python program to read the API (<https://dummyjson.com/users>) and perform the following tasks:
 - a. Display result
 - b. Sort the result by country
 - c. Sort the result by Age > 28
5. Write a class-based iterator that returns the **Fibonacci sequence** up to ***n* numbers**.

6. Create an iterator that takes a list and iterates over it in reverse order.
7. Write a generator function ***even numbers(n)*** that yields even numbers up to n.
8. Create a generator that yields an infinite sequence of prime numbers.
9. Use a generator expression to find the sum of squares of numbers from 1 to 100.
10. Write a decorator that logs the execution time of a function.
11. Create a decorator that checks if a user is authenticated before allowing function execution.
12. Write a decorator that repeats a function ***n*** times.
13. Create a coroutine that receives strings and prints only those containing a specific keyword.
14. Create a coroutine that collects numbers and calculates the average. Stop on a custom exception.

SESSION-8 (Based on MCS-081: Unit 13)

1. Write a Python program to create a 1D, 2D, and 3D NumPy array and display the shape and dimension of each array.
2. Generate a random array of 15 integers between 10 and 200 and find the following:
 - a. Maximum, minimum, sum, and mean
 - b. Standard deviation and variance
3. Write a Python program to perform two Array concatenations and Splitting:
 - a. Concatenate:
 - i. Horizontally (column-wise)
 - ii. Vertically (row-wise)
 - b. Split the result back into the original arrays.
4. Create a NumPy array of 20 random integers between 1 and 100.
 - a. Filter and display elements greater than 50
 - b. Replace all even numbers with -1
5. Create two 4×4 matrices using Linear Algebra with NumPy and perform the following tasks:
 - a. Perform matrix multiplication
 - b. Find transpose and inverse (if invertible)
 - c. Use np.dot () and np.linalg.inv ()
6. Create an array of shape (4×4) and add a 1D array [1, 2, 3, 4] to it using broadcasting. Explain how broadcasting works in this case.

SESSION-9 (Based on MCS-081: Unit-14 and Unit 15)

1. Write a Python program using Pandas to load a CSV file (e.g., students.csv) into a DataFrame. Display the first 5 rows, dataset shape, column names, data types, and basic statistics.
2. Write a Python program using Pandas to Data Cleaning and Handling Missing Values. Use a student's dataset with missing values, perform:
 - a. Counting nulls
 - b. Dropping rows with nulls
 - c. Filling nulls with the mean/median/mode
3. Write a Python program using Pandas to Merge two DataFrames on a common key using different types of joins (inner, left, right, outer). use any dataset of student, course and result dataset.
4. Practice key characteristic of DataFrame in Pandas, using Common DataFrame operations, listed below (<https://pandas.pydata.org/docs/reference/frame.html>):
 - a. Constructor
 - b. Attributes and underlying data
 - c. Conversion
 - d. Indexing, iteration
 - e. Binary operator functions
 - f. Function application, GroupBy & window
 - g. Computations / descriptive stats
 - h. Reindexing / selection / label manipulation
 - i. Missing data handling
 - j. Reshaping, sorting, transposing
 - k. Combining / comparing / joining / merging
 - l. Time Series-related
 - m. Flags
 - n. Metadata and plotting
 - o. Serialization / IO / conversion
 - p. Sparse accessor

SESSION-10 (Based on MCS-081: Unit-16)

1. Create a pie chart for gender and category analysis in the election separately.
2. Create a bar chart for votes received to all parties in the election.
3. Create two different charts (e.g., line and bar chart) using **plt.subplot ()** to display them in the same window (side by side or top-bottom).
4. Create a scatter, histogram, and other charts for more practice.
5. refer the official documentation of Matplotlib
https://matplotlib.org/stable/plot_types/index.html

1.5 REFERENCES / FURTHER READINGS

- Download Python Setup: <https://www.python.org/downloads/>
- Instructions to install Python for various platforms: <https://docs.python.org/3/using/index.html>
- Download VSCode IDE for various platforms: <https://code.visualstudio.com/download>
- Download PyCharm IDE for various platforms: <https://www.jetbrains.com/pycharm/>
- Official NumPy Website: <https://numpy.org/>
- NumPy Document: <https://numpy.org/doc/>
- Pandas Official Website: <https://pandas.pydata.org/>
- Pandas Document: https://pandas.pydata.org/docs/user_guide/index.html
- DataFrame: <https://pandas.pydata.org/docs/reference/frame.html>
- Matplotlib Documentation : <https://matplotlib.org/stable/tutorials/pyplot.html>



ignou
THE PEOPLE'S
UNIVERSITY